beforehand. Team members should not require approval where post-monitoring actions and feedback can work. More importantly, proactive team members should be recognised and developers should be encouraged to network, so that they can have a broader perspective of the organisation.

Eliminating spoon-feeding and encouraging new developers to do their own research can go a long way in building their confidence. Every member of the team should have clear instructions as to what they are accountable for and why they are an integral part of the project. Developers should also be clearly told what is mandatory and what is not, so that they can prioritise their work.

Most importantly, face-to-face communication over video calls or in person should be encouraged wherever possible. In these meetings, experienced developers should ask questions to specific developers instead of generally floating a question to the entire team.

7. Eliminating waste

'Eliminating waste' consists of seven categories, each of which is mentioned below.

Partially done work: This type of work is done haphazardly without any concern for the definition of *Done* (discussed earlier in build quality). This work, as a result of this bad practice, cannot be used by the development team in the final software, and thus the manpower and the resources used in making the module are wasted. Correctly defining *Done* can eliminate this waste.

Extra features: The Pareto principle or the 80/20 rule implies that 80 per cent of users will most likely end up using only 20 per cent of the features. The MoSCoW method is a type of prioritisation technique used widely in product management, where each feature is categorized into *Must have*, *Should have*, *Could have* and *Will not have*. This solidifies the priority of each feature and when it should be implemented.

Relearning/extra processing: Relearning occurs when different members of the team have to go on learning or creating a task/work that has already been done by some other member of the team. Seeing the project as a whole (discussed earlier in 'optimising the whole') can help with feature and module relearning. Other practices like daily stand ups and weekly meetings can help the team to get in sync with all the work being done for the project.

Handoffs: Handoffs occur whenever work is passed from one role to another. Another seemingly simple solution is to minimise the number of handoffs necessary to complete one full iteration — from the analyst to the tester. This can be done by making cross-functional teams. Product-oriented teams have other distinct advantages because they also share a common goal, which helps with another lean principle of 'optimising the whole'.

Delays: Delays are anything that act as a hindrance to the commencement of a value adding process or elongate it. If team members don't know what to do, they can either sit idle or switch to other tasks, either of which results in a loss of flow. This leads to partially done work. Delays can also extend the duration of feedback loops. To deal with delays, smaller iterations should be adopted as they require shorter deadlines, leading to faster feedback loops.

Task switching: We only have to look at one word to understand why task switching can be an issue that creates waste, i.e., flow. Every team member and, more importantly, programmer needs a free flowing chain of thought to optimise output. It takes a good amount of time to get into this flow, but it can be broken with the slightest disturbance or lateral thoughts. Jeff Attwood, the founder of StackOverflow, has said that by the time you add a third project to the mix, nearly half of the time is wasted in task switching. Mixing up fewer projects at a time is the best way to avoid this waste.

Defects: This one needs no definition because it is self-evident. However, what we can define is the waste created by a particular defect. This is given as: *Waste = (Impact) x (Time)*. We can reduce defects by antibody tests. Whenever a defect is detected, the first step should not be to remove it. Instead, the first task should be to write a test which would fail in the presence of the newly found defect. This test should be added to the archives of tests. Whenever another developer reintroduces the same defect, it will be detected by that test.

Lean software development is a new practice that focuses on higher quality products made on the principles of efficiency, innovation and social empowerment. Principles like amplified learning and deciding as late as possible fuel innovation in a project, which is complemented by delivering as fast as possible and building integrity, focusing on the implementation of these innovative ideas and building a viable product. The principles of optimising the whole and empowering the team are focused on overall empowerment of the open source software developers, project owners, users and all the other stakeholders a project can have.

Lean principles are evolving and being refined constantly. The intertwined and complementary nature of these principles makes them very successful when managing any project. END

By: Aakash Shah and Vishva Patel

Aakash Shah is currently exploring the field of deep learning and, more specifically, computer vision. He also likes competitive programming and often solves problems on HackerRank and Leetcode.

Vishva Patel is deeply interested in the world of open source software development and software product management. He is determined to create software solutions using the development skills he tries to acquire proactively.